
ForL0 State Backend

使用说明书

2026 年 8 月

目 录

1	产品概述	1
1.1	关键特性	1
1.2	适用范围	1
1.3	支持的状态类型	1
2	启用与部署	2
2.1	产物部署	2
2.2	集群配置	2
2.3	加载校验	3
3	配置参考	3
3.1	启用后端	3
3.1.1	配置文件方式（推荐）	3
3.1.2	代码方式	3
3.2	配置项列表	4
3.3	L0 HotCache 启用条件	5
3.4	SwissTable 容量参数	5
4	作业中的状态使用	5
4.1	DataStream API	6
4.2	Flink SQL 与 Table API	6
4.3	命名空间行为	7
5	Checkpoint 与 Savepoint	8
5.1	Checkpoint 配置	8
5.2	Savepoint 触发与恢复	8
5.3	兼容性约定	8
6	作业提交	8
6.1	Flink CLI	9
6.2	REST API	9

7	运行监控	9
7.1	日志约定	9
7.2	运行期加载检查 API	10
7.3	HotCache 指标	10
8	调优建议	11
8.1	并行度与 KeyGroup	11
8.2	L0 Cache 容量	11
8.3	TaskManager 内存	12
8.4	Checkpoint 间隔	12
9	常见问题	13

1 产品概述

ForL0 State Backend 是面向 Apache Flink 的高性能 Keyed State 后端，针对鲲鹏服务器的体系结构特性进行了定向优化。系统以 C++ 原生引擎承担状态的权威存储与访问，以 Java 控制层与 JNI 桥接层承担 Flink 接口适配与状态注册，并通过 SwissTable 哈希结构、SWAR 并行匹配以及可选的 L0 Cache 加速缩短热点状态访问路径。

本文档内容限定在状态后端的配置、作业集成、运行监控与调优。底层架构细节由《设计说明书》给出，编译与部署由《编译构建指导书》给出。

1.1 关键特性

- 支持本项目已验证的 Flink Keyed State 接口与语义，现有作业代码无需改动。
- 原生 SwissTable 哈希结构 + NEON SWAR 并行匹配，针对鲲鹏 ARM64 做指令级优化。
- 可选启用鲲鹏 L0 Cache 作为热键访问的上层快路径，硬件条件不满足时自动保持关闭，底层 SwissTable 直接接管状态访问。
- 支持 Flink Checkpoint 与 Savepoint，采用完整 KeyGroup 快照和恢复。
- 原生 Copy-On-Write 一致性视图，快照数据在异步阶段写出。

1.2 适用范围

项目	要求
处理器	鲲鹏系列处理器
操作系统	openEuler
JDK	JDK 1.8 及以上 (aarch64)，建议使用华为毕昇 JDK
Flink	Apache Flink 1.20.3
运行模式	标准模式 / L0 HotCache 模式 (需 /dev/hisi_l0 或 /dev/l0，以及 libl0mempool.so)

1.3 支持的状态类型

ForL0 State Backend 对以下五类常用 Keyed State 提供原生存储实现：

状态类型	用途	实现类
ValueState	单值键控状态	ForL0ValueState
ListState	列表键控状态	ForL0ListState
MapState	键控内部映射状态	ForL0MapState
ReducingState	归约键控状态	ForL0ReducingState
AggregatingState	聚合键控状态	ForL0AggregatingState

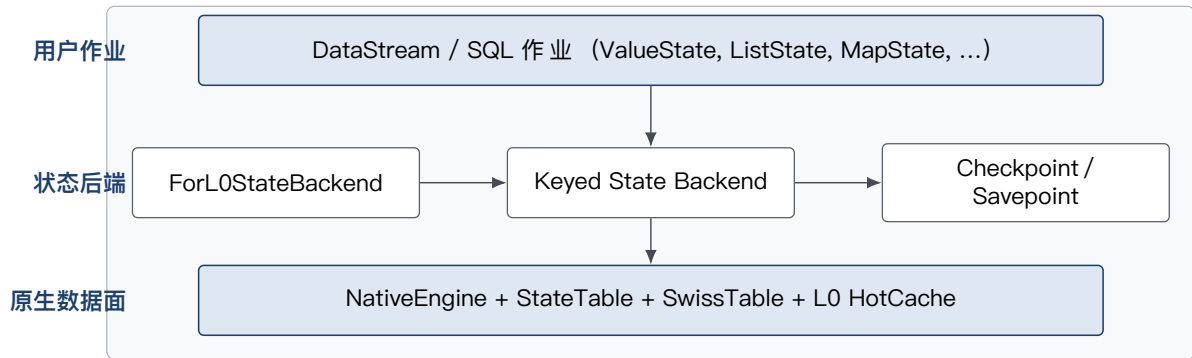


图 1 用户作业与 ForL0 State Backend 的运行时集成关系

2 启用与部署

本节假设 ForL0 State Backend 的 JAR 与原生库已按《编译构建指导书》完成构建。启用流程主要包括将产物放置到 Flink 对应目录、追加配置并重启集群。

2.1 产物部署

```

# 1. 状态后端 JAR
cp flink-statebackend-forL0-1.0-SNAPSHOT.jar $FLINK_HOME/lib/

# 2. 原生引擎共享库
mkdir -p $FLINK_HOME/native
cp libforl0_engine.so $FLINK_HOME/native/

# 3. 多节点集群下，将上述两份文件同步到所有 TaskManager 节点
  
```

2.2 集群配置

在 \$FLINK_HOME/conf/config.yaml 中追加如下配置项：

```

# 启用 ForL0 作为默认状态后端（使用 StateBackendFactory 的全限定类名）
state:
  backend:
    type: org.apache.flink.state.forl0.ForL0StateBackendFactory
  checkpoints:
    dir: file:///data/flink/checkpoints
    checkpoint-storage: filesystem

# 指定原生库路径（针对 TaskManager 进程）
env:
  java:
    opts:
      all: -Djava.library.path=/opt/flink-1.20.3/native
  
```

配置完成后重启集群：

```
$FLINK_HOME/bin/stop-cluster.sh  
$FLINK_HOME/bin/start-cluster.sh
```

2.3 加载校验

集群启动后，可通过 TaskManager 日志确认状态后端与原生引擎是否正确加载：

```
grep "[ForL0]" $FLINK_HOME/log/flink-*-taskexecutor-*.log
```

输出中至少应包含原生库加载成功一条记录；如已启用 `l0-cache.enabled=true`，还会在 KeyedStateBackend 创建时输出一条包含 `l0Enabled=true/false` 的引擎创建记录：

```
[ForL0] Native engine loaded via java.library.path  
[ForL0] C++ engine created: handle=..., keyGroups=[0, 8), total=128,  
      l0Enabled=true
```

若日志中的 `l0Enabled=false`，则表示运行环境不满足 L0 硬件门禁或配置未启用，HotCache 快路径未激活，状态访问仍由 SwissTable 正常服务。

3 配置参考

ForL0 State Backend 的配置分为两类：Flink 标准状态后端配置和 ForL0 State Backend 自身引入的 `state.backend.forl0.*` 专用配置。两类配置都可在集群配置文件中声明，也可在作业提交时通过 `-D` 参数覆盖。

3.1 启用后端

3.1.1 配置文件方式（推荐）

在集群配置文件中声明默认状态后端后，全集群作业自动生效。`state.backend.type` 必须填写 `StateBackendFactory` 的全限定类名：

```
state:  
  backend:  
    type: org.apache.flink.state.forl0.ForL0StateBackendFactory
```

3.1.2 代码方式

在作业中显式设置，仅对当前作业生效：

```
StreamExecutionEnvironment env =
```

```
StreamExecutionEnvironment.getExecutionEnvironment();
env.setStateBackend(new ForL0StateBackend());
```

若需通过 Factory 由配置驱动创建，可使用：

```
env.setStateBackend(
    new ForL0StateBackendFactory().createFromConfig(
        new Configuration(),
        Thread.currentThread().getContextClassLoader()));
```

3.2 配置项列表

配置项	类型	默认值	说明
state.backend.type	String	-	填写 StateBackendFactory 的全限定类名以启用本后端，具体值见 § 3.1。
state.backend.forl0.async-snapshots	boolean	true	使用原生 Copy-On-Write 异步快照。
state.backend.forl0.l0-cache.enabled	boolean	false	是否启用 L0 Cache 加速，需鲲鹏硬件支持。
state.backend.forl0.l0-cache.size	MemorySize	20 mb	兼容配置项，表示单个 TaskManager 进程的 L0 配额。
state.backend.forl0.l0-cache.total-size	MemorySize	-	主机/设备级 L0 总预算；按 expected-engines 划分给 TaskManager 进程。
state.backend.forl0.l0-cache.expected-engines	int	1	预期进程级 manager 数；应填写 TaskManager 进程数，不是 slot 或 StateEngine 数。
state.backend.forl0.l0-cache.strict-allocation	boolean	false	为 true 时，硬件门禁或分配失败直接使作业失败，适用于要求 L0 必须生效的部署。
state.backend.forl0.l0-cache.state-size	MemorySize	1 mb	每个符合条件的标量 ValueState 请求的缓存配额。
state.backend.forl0.l0-cache.write-bypass-threshold	long	1,048,576	连续写入达到该次数且无读取时暂停缓存准入；0 表示禁用该策略。
state.backend.forl0.native-memory.max-size	MemorySize	0	单 StateEngine 的 SwissTable native 内存上限；0 表示不设限。
forl0.metricsCollector.enabled	boolean	false	是否注册 HotCache gauge；需要监控缓存状态时显式开启。
state.backend.forl0.initial-table-capacity	int	64	单张 SwissTable 的初始 slot 数，必须为 2 的幂且 ≥ 8 。
state.backend.forl0.max-table-capacity	int	0	单张 SwissTable 的容量上限；0 表示不设上限。
state.backend.forl0.main-table.load-factor-threshold	double	0.875	SwissTable 扩容阈值，取值范围为 0.5 - 0.875。

3.3 L0 HotCache 启用条件

当 `l0-cache.enabled` 配置为 `true` 时，原生引擎在运行期继续检查以下三项：

1. 共享库 `libl0mempool.so` 可通过 `dlopen` 加载（依次检查 `libl0mempool.so`、`/usr/lib`、`/usr/lib64`、`/lib`、`/lib64`）。
2. L0 设备节点 `/dev/hisi_l0` 或 `/dev/l0` 存在且可访问。
3. 首次探测性 `l0_mem_alloc` 调用成功。

任一项不满足时，非 `strict` 作业会禁用 HotCache，状态访问仍由底层 `SwissTable` 正常服务；`strict` 作业则直接失败，保证要求 L0 的部署不会在缓存未激活时继续运行。运行时应结合日志与指标确认：`L0 cache disabled by config` 表示配置关闭；`L0 hardware not available ... forcibly disabled` 表示硬件门禁失败；`engine_start active=1` 表示当前 TaskManager 首次成功创建进程级 manager；`engine_reuse active=1` 表示同进程后续 StateEngine 正在复用该 manager；`state_attach` 表示受支持状态已取得 HotSet（见 § 7.1）。

每个 TaskManager 进程只持有一个共享 HotCacheManager。因此 `l0-cache.total-size` 的预算划分按 TaskManager 进程数计算；提高 operator 并行度会增加共享池中的 state cache 数量，但不会再重复创建硬件 tuner。不同 TaskManager 之间仍是独立进程和独立 manager。

3.4 SwissTable 容量参数

以下参数决定单张 SwissTable 的隐含容量范围，默认值适合绝大多数作业，仅在超大规模或调优时才需要修改：

配置项	默认值	说明
<code>state.backend.forl0.initial-table-capacity</code>	64	初始 slot 数，必须为 2 的幂且 ≥ 8 。
<code>state.backend.forl0.max-table-capacity</code>	0	单表容量上限；0 表示不设上限，非零值必须为 2 的幂。
<code>state.backend.forl0.main-table.load-factor-threshold</code>	0.875	触发 rehash / grow 的负载阈值，可在 0.5 - 0.875 范围内配置。

4 作业中的状态使用

ForL0 State Backend 使用 Flink 标准状态接口。用户作业中状态的声明、读写和清理方式与使用 `HashMapStateBackend` 时一致。

4.1 DataStream API

以下示例在 `KeyedProcessFunction` 中维护单值状态。切换到 `ForL0 State Backend` 不需要对代码作任何修改：

```
StreamExecutionEnvironment env =
    StreamExecutionEnvironment.getExecutionEnvironment();

// 方式一：集群配置已将 state.backend.type 设为 org.apache.flink.state.forl0.
// ForL0StateBackendFactory，无需代码改动
// 方式二：在作业中显式设置
env.setStateBackend(new ForL0StateBackend());

input
    .keyBy(t -> t.f0)
    .process(new KeyedProcessFunction<String, Tuple2<String, Long>,
String>() {
        private ValueState<Long> countState;

        @Override
        public void open(Configuration parameters) {
            countState = getRuntimeContext().getState(
                new ValueStateDescriptor<>("count", Long.class));
        }

        @Override
        public void processElement(Tuple2<String, Long> value,
            Context ctx,
            Collector<String> out) throws
Exception {
            Long current = countState.value();
            current = (current == null) ? 1L : current + 1;
            countState.update(current);
            out.collect(value.f0 + ": " + current);
        }
    });
```

4.2 Flink SQL 与 Table API

SQL 作业通过集群配置启用 `ForL0 State Backend` 即可，运行方式与使用其他状态后端时没有差别。例如：

```
CREATE TABLE orders (
    order_id BIGINT,
    user_id  BIGINT,
```

```

amount    DOUBLE,
order_time TIMESTAMP(3),
WATERMARK FOR order_time AS order_time - INTERVAL '5' SECOND
) WITH (...);

SELECT
  user_id,
  TUMBLE_START(order_time, INTERVAL '1' MINUTE) AS window_start,
  SUM(amount) AS total_amount
FROM orders
GROUP BY user_id, TUMBLE(order_time, INTERVAL '1' MINUTE);

```

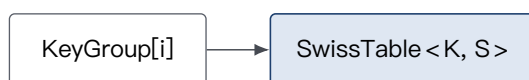
SQL 作业会使用已配置的 ForL0 State Backend。具体状态访问路径由运行时 serializer、key 和 namespace 类型决定，无需在 SQL 中增加 ForL0 专用语法。

4.3 命名空间行为

ForL0 State Backend 的状态组织区分命名空间类型，对不同场景给出不同的访问路径：

- **VoidNamespace 直通路径：**在非窗口算子中（例如 KeyedProcessFunction 中的 ValueState），命名空间恒为 VoidNamespace。ForL0 State Backend 检测到这种情形后直接访问当前 KeyGroup 对应的 SwissTable，跳过命名空间容器，避免额外的容器查找开销。
- **TimeWindow 命名空间路径：**在窗口算子中，以 TimeWindow 作为命名空间的状态由每个命名空间独立的 SwissTable 承载。命名空间之间隔离，避免 key 冲突。
- **空命名空间回收：**当某个命名空间下的 SwissTable 因清理操作变为空时，ForL0 State Backend 会将该命名空间条目从容器中移除，避免无效命名空间持续占用内存。

VoidNamespace 路径



TimeWindow 命名空间路径



图 2 ForL0 State Backend 的命名空间访问路径

5 Checkpoint 与 Savepoint

ForL0 State Backend 使用原生 Copy-On-Write 机制建立一致性视图，并通过 Flink Checkpoint 框架异步写出。用户侧的配置方式和操作方式保持不变。

5.1 Checkpoint 配置

```
execution:
  checkpointing:
    interval: 60000          # 60 秒触发一次
    mode: EXACTLY_ONCE
    externalized-checkpoint-retention: RETAIN_ON_CANCELLATION

state:
  checkpoints:
    dir: file:///data/flink/checkpoints
  savepoints:
    dir: file:///data/flink/savepoints
```

5.2 Savepoint 触发与恢复

可通过 Flink CLI 触发 Savepoint 与恢复：

```
# 为运行中的作业触发 Savepoint
$FLINK_HOME/bin/flink savepoint <jobId> file:///data/flink/savepoints

# 从指定 Savepoint 启动作业
$FLINK_HOME/bin/flink run \
  -s file:///data/flink/savepoints/savepoint-xxx \
  -d your-job.jar
```

5.3 兼容性约定

- 从 ForL0 State Backend 产出的 Checkpoint 或 Savepoint, 可在同一版本的 ForL0 State Backend 之间互相恢复。
- 从其他状态后端迁移到 ForL0 State Backend 时，应使用 canonical Savepoint，并在测试环境验证状态类型、serializer 与定时器恢复结果。其他后端生成的 Checkpoint 不作为跨后端迁移入口。
- 进行跨后端迁移前，建议在同等输入的测试环境下对账一次状态清单，以降低升级风险。

6 作业提交

6.1 Flink CLI

```
$FLINK_HOME/bin/flink run -d \  
-Dstate.backend.type=org.apache.flink.state.forl0.  
ForL0StateBackendFactory \  
-Dstate.backend.forl0.l0-cache.enabled=true \  
-p 8 \  
your-application.jar
```

如已在集群配置中将 `state.backend.type` 设为 `org.apache.flink.state.forl0.ForL0StateBackendFactory`, 命令行的 `-Dstate.backend.type` 可省略。

6.2 REST API

```
# 上传 JAR  
curl -X POST -H "Expect:" \  
-F "jarfile=@your-application.jar" \  
http://<jobmanager>:8081/jars/upload  
  
# 提交作业并覆盖状态后端配置  
curl -X POST http://<jobmanager>:8081/jars/<jar-id>/run \  
-d '{  
  "programArgs": "--parallelism 8",  
  "flinkConfiguration": {  
    "state.backend.type": "org.apache.flink.state.forl0.  
ForL0StateBackendFactory"  
  }  
'
```

7 运行监控

7.1 日志约定

ForL0 State Backend 相关的运行期日志一律以 [ForL0] 前缀输出。常见条目见下表:

日志	含义
[ForL0] Native engine loaded via java.library.path	通过 java.library.path 加载成功
[ForL0] Native engine loaded from bundled resource: ...	从 JAR 内置资源解压并加载成功
[ForL0] Creating ForL0StateBackend from configuration.	通过 Factory 从配置创建后端
[ForL0] Building ForL0KeyedStateBackend with C++ engine...	KeyedStateBackend 构造开始
[ForL0] C++ engine created: handle=..., l0Enabled=...	C++ 引擎创建完成, l0Enabled 反映 HotCache 是否激活
[ForL0-HotCache] L0 cache disabled by config.	当前配置未启用 HotCache
[ForL0-HotCache] engine_start active=1 ...	当前 TaskManager 首次成功初始化进程级 L0 manager
[ForL0-HotCache] engine_reuse active=1 ...	同一 TaskManager 内其他 StateEngine 复用已有 manager, 属于正常并行行为
[ForL0-HotCache] state_attach name=...	受支持状态已从共享池取得 HotSet
[ForL0-HotCache] WARN: L0 hardware not available ...	配置要求 L0, 但设备、库或分配门禁失败
[ForL0] ForL0KeyedStateBackend built successfully.	KeyedStateBackend 构造完成
[ForL0] Failed to load native engine: ...	原生库加载失败, 后端不会启动

7.2 运行期加载检查 API

在用户代码或自检脚本中判断原生引擎是否已加载:

```
import org.apache.flink.state.forl0.NativeEngine;

boolean loaded = NativeEngine.isLoaded();
NativeEngine.ensureLoaded(); // 未加载时抛出 RuntimeException
```

原生引擎本身在所在类被容器加载时完成一次性的 System.loadLibrary("forl0_engine"), 不提供重新初始化接口。

7.3 HotCache 指标

启用 L0 Hot-Key Cache 且设置 forL0.metricsCollector.enabled=true 时, 后端在作业的 MetricGroup 下注册以下 gauge, 指标名为 <operator>.forl0.hotcache.<field>,

可直接在 Flink Web UI 或对接的指标后端（Prometheus / InfluxDB 等）中观察：

字段	含义
active	HotCacheManager 是否真正激活（1 表示硬件门禁通过；0 表示未激活）
bytesCapacity	L0 分区从 <code>l0_mem_alloc</code> 实际取得的总容量，单位为字节
bytesUsed	当前 HotSet 已用字节数
totalSets	全局 HotSet 总数
freeSets	当前可用的 HotSet 数
lookups	HotCache 查询总次数
hits	HotCache 命中总次数（命中率由 <code>hits / lookups</code> 给出）
invalidations	HotCache 失效总次数（erase / 覆写触发）
bytesRequested	当前 TaskManager 请求的 L0 配额，单位为字节
writes	HotCache 写入总次数
writeBypassEvents	连续写入触发缓存准入暂停的次数
stateCaches	当前已附着的状态缓存数量

当设置 `l0-cache.enabled=true` 但 `active` 始终为 0 时，说明硬件门禁未通过（见 § 3.3），非 strict 作业已回退到纯 SwissTable 路径；此时 `bytesCapacity=0`、`hits=0`、`lookups=0`。若 `active=1` 且存在 `state_attach`，但 `lookups=0`，表示缓存已附着但当前负载没有执行缓存读取，命中率应记为“不适用”而不是 0%。

8 调优建议

8.1 并行度与 KeyGroup

ForL0 State Backend 依照 Flink 标准的 KeyGroup 路由模型组织状态。建议并行度设置与实际可用 slot 数对齐，避免因并行度过高导致的 KeyGroup 过细、快照开销相对增大的问题。鲲鹏服务器常见的推荐公式为：

$$\text{parallelism} = \text{TaskManager 数量} \times \text{每 TM 的 slot 数}$$

8.2 L0 Cache 容量

开启 L0 Cache 的作业可参考以下容量范围。实际分配成功的字节数由 `forl0.hotcache.bytesCapacity` 反映：

配置场景	建议配置	说明
单 TaskManager	<code>l0-cache.size</code> : 20 mb	使用兼容配置项设置当前进程配额。
多 TaskManager	<code>l0-cache.total-size</code> + <code>expected-engines</code>	按设备可用预算设置总量，并按 TaskManager 进程数均分。
单状态缓存	<code>l0-cache.state-size</code> : 1 mb	仅符合类型条件的标量 ValueState 会申请该配额。

8.3 TaskManager 内存

ForL0 State Backend 的 Keyed KV 状态位于原生内存，优先队列仍由 Flink 管理；本后端不使用 Flink Managed Memory 保存 KV 状态。据此可作如下调整：

- 适当增加 `taskmanager.memory.process.size` 的上限，以容纳原生状态数据。
- 相应降低 `taskmanager.memory.managed.size`，为原生数据面让出空间。
- 若开启 L0 Cache，确保 `l0-cache.total-size` 与设备实际可用预算匹配，并正确设置 TaskManager 进程数。

8.4 Checkpoint 间隔

ForL0 State Backend 通过 Copy-On-Write 缩短同步准备阶段，快照开销主要来自变更记录、序列化与写出。建议：

- 保持 `state.backend.forl0.async-snapshots`: `true`。
- 根据容灾目标设置 `execution.checkpointing.interval`，无特殊需求时使用 30 – 60 秒量级。
- 生产环境将 Checkpoint 目录指向稳定的共享存储（HDFS、NAS 等）。

9 常见问题

现象	可能原因	处理方式
UnsatisfiedLinkError	原生库路径未配置	检查 <code>java.library.path</code> 是否包含 <code>libforl0_engine.so</code> 所在目录
ClassNotFoundException	JAR 未部署	确认 <code>\$FLINK_HOME/lib/</code> 下存在 <code>flink-statebackend-forL0-*.jar</code>
日志中 <code>l0Enabled=false</code>	L0 硬件/库不可用或未启用	核查 <code>/dev/hisi_l0</code> 或 <code>/dev/l0</code> 、 <code>libl0mempool.so</code> 以及 <code>l0-cache.enabled</code>
Checkpoint 失败	Checkpoint 目录权限不足	确认 <code>state.checkpoints.dir</code> 路径对 Flink 进程可写
<code>forl0.hotcache.active=0</code> 但配置已开启	硬件门禁未通过	检查 L0 设备节点是否存在, <code>libl0mempool.so</code> 是否在搜索路径上
配置要求 L0, 但日志出现 <code>disabled by config</code>	配置未生效或被作业参数覆盖	检查集群配置与作业提交参数中的 <code>l0-cache.enabled</code>
只有 <code>engine_start</code> 后又有多个 <code>engine_reuse</code>	同一 TaskManager 内并行 StateEngine 共享 manager	正常行为; 继续确认存在 <code>state_attach</code> , 且无 <code>hardware-not-available</code> 警告
作业 OOM	状态规模超出进程内存预算	增加 <code>taskmanager.memory.process.size</code> , 或设置状态 TTL